



Comment fonctionne un émulateur Super Nintendo

Guide pédagogique complet — du transistor au pixel, du processeur au son

Émulateur écrit **from scratch en Rust** — CPU 65C816, PPU graphique, APU sonore (SPC700 + S-DSP), DMA/HDMA, cartouches LoROM/HiROM et coprocesseur SuperFX.
Document technique et pédagogique · Version 2.0 — édition détaillée

Table des matières

1. Qu'est-ce qu'émuler une console ?
2. Petit rappel : bits, octets et hexadécimal
3. Vue d'ensemble : qui parle à qui
4. La cartouche et la carte mémoire
5. Le cœur : le processeur 65C816
6. Le chef d'orchestre : le timing et l'ordonnanceur
7. La puce graphique : le PPU
8. Le déménageur : DMA et HDMA
9. Le son : l'APU (SPC700 + S-DSP)
10. Les puces de cartouche : le coprocesseur SuperFX
11. La boucle d'émulation : tout synchroniser
12. Au-delà de la console : sauvegardes et save states
13. Comment cet émulateur a été construit et vérifié
14. Glossaire

1. Qu'est-ce qu'émuler une console ?

Une console de jeu comme la Super Nintendo (SNES / Super Famicom, sortie en 1990) est un ensemble de **puces électroniques** qui coopèrent : un processeur, une puce vidéo, une puce audio, de la mémoire. **Émuler** la console, c'est écrire un programme qui **imite le comportement de chacune de ces puces**, assez fidèlement pour que le code d'origine d'un jeu — les mêmes octets que sur la cartouche — s'exécute et produise exactement la même image et le même son que sur la vraie machine.

💡 Analogie

Émuler, ce n'est pas « refaire le jeu ». C'est construire une **fausse console** si convaincante que la vraie cartouche ne voit pas la différence. Le jeu croit parler à une vraie puce vidéo ; en réalité il parle à quelques milliers de lignes de Rust qui répondent exactement comme le silicium d'origine l'aurait fait.

Pourquoi la fidélité est difficile

Le défi n'est pas d'imiter une puce isolée : c'est de faire **cohabiter toutes les puces au rythme exact**. Les programmeurs de jeux des années 90 exploitaient le moindre détail de synchronisation — par exemple changer une couleur pile au moment où le canon à électrons du téléviseur passe à la ligne suivante, pour obtenir un dégradé. Si l'émulateur se trompe de quelques cycles, l'effet casse. Un bon émulateur doit donc reproduire non seulement *ce que* font les puces, mais *quand* elles le font.

La SNES est une machine **16 bits**. Ses composants principaux :

- **Le CPU 65C816** — le cerveau : il exécute la logique du jeu (physique, IA, score...).
- **Le PPU** (*Picture Processing Unit*) — la puce graphique : elle compose l'image, ligne par ligne.
- **L'APU** (*Audio Processing Unit*) — un mini-ordinateur sonore complet et autonome.
- **La mémoire** — 128 Ko de travail (WRAM), 64 Ko de vidéo (VRAM), et la cartouche (ROM + sauvegarde).
- **Le DMA** — un « déménageur » d'octets ultra-rapide entre ces mémoires.
- **Parfois, une puce dans la cartouche** — comme le SuperFX, un second processeur qui dessine de la 3D.

2. Petit rappel : bits, octets et hexadécimal

Pour comprendre la suite, trois notions suffisent. Tout, dans une console, se ramène à des **bits** — des interrupteurs qui valent 0 ou 1.

- Un **octet** (*byte*) = 8 bits. Il représente un nombre de 0 à 255.
- Un **mot** (*word*) 16 bits = 2 octets, de 0 à 65 535.
- On note souvent les nombres en **hexadécimal** (base 16), préfixés par **\$**. Chaque chiffre hexa vaut 4 bits. Ainsi **\$FF** = 255, **\$8000** = 32768.

Un octet valant `$A9` (169), c'est ces huit interrupteurs :



$128 + 32 + 8 + 1 = 169 = \text{\$A9}$. (C'est aussi, par coïncidence, l'opcode de l'instruction « charge l'accumulateur » du 65C816 — on le reverra.)

🔑 À retenir

Un **registre** est une petite case mémoire à l'intérieur d'une puce. Écrire une valeur dans un registre revient à **appuyer sur des boutons de commande** de la puce. La quasi-totalité du dialogue entre le CPU et les autres puces se fait ainsi : en écrivant des octets à des adresses spéciales appelées **registres MMIO**.

3. Vue d'ensemble : qui parle à qui

Tous les composants sont reliés par un **bus** : une autoroute d'adresses par laquelle le CPU lit et écrit. Quand le CPU accède à une adresse, ce n'est pas toujours de la mémoire — cela peut être un registre qui commande le PPU, l'audio, ou un transfert. Le rôle du bus est de **router** chaque accès vers le bon destinataire, et d'avancer l'horloge du bon nombre de cycles.



🔑 À retenir

Dans le code de l'émulateur, cette structure se retrouve telle quelle : un module `cpu`, un module `ppu`, un module `apu`, un `bus` qui les relie, et un `scheduler` (l'ordonnanceur/horloge) qui garde tout le monde synchronisé. Le cœur d'émulation est une bibliothèque **sans aucune entrée/sortie** : elle se teste toute seule, sans écran ni son. Un programme séparé (le « frontend ») ouvre la fenêtre, joue le son et lit le clavier.

La séparation cœur / frontend

Cette séparation est un principe d'ingénierie important. Le **cœur** ne connaît ni le clavier, ni la carte son, ni le système d'exploitation : il transforme simplement « appuis de manette + temps qui passe » en « image + échantillons audio ». Le **frontend** traduit cela vers le monde réel. Résultat : on peut faire tourner le cœur en mode « sans tête » (headless) pour le tester automatiquement — c'est exactement comme cela que cet émulateur a été vérifié (voir §13).

4. La cartouche et la carte mémoire

Le 65C816 adresse la mémoire sur **24 bits** : 16 Mo d'espace, découpé en 256 « banques » (*banks*) de 64 Ko. Une adresse s'écrit `banque:offset`, par exemple `$7E:0100`. Mais toutes les adresses ne mènent

pas à de la RAM : selon la zone, un même accès touche la mémoire de travail, une puce, ou la cartouche.

\$00–\$3F : \$0000	Miroir des 8 premiers Ko de la WRAM
\$2100–\$213F	Registres du PPU (vidéo)
\$2140–\$2143	Ports de l'APU (son)
\$4200–\$421F	Registres CPU internes (NMI, IRQ, manettes)
\$4300–\$437F	Registres des 8 canaux DMA
\$8000–\$FFFF	ROM de la cartouche
\$70–\$7D : \$0000	Sauvegarde SRAM (pile de la cartouche)
\$7E–\$7F : entier	128 Ko de WRAM (mémoire de travail)

LoROM et HiROM : deux câblages

Les cartouches câblent leur ROM de deux façons principales. En **LoROM**, la ROM est exposée par blocs de 32 Ko dans la moitié haute (\$8000–\$FFFF) de chaque banque. En **HiROM**, elle occupe des banques pleines de 64 Ko. L'émulateur ne le sait pas d'avance : il lit l'**en-tête** interne de la ROM (un petit bloc de métadonnées à une position fixe) et attribue un **score** à chaque hypothèse — le couple somme de contrôle / complément doit valoir \$FFFF , le titre doit être lisible, l'octet de mapping doit être plausible — puis retient la meilleure. Il en déduit aussi la **région** (PAL 50 Hz ou NTSC 60 Hz) et la taille de la sauvegarde.

Pour aller plus loin

L'open bus, un détail subtil mais crucial. Sur le vrai matériel, lire une adresse « qui ne mène nulle part » ne renvoie pas 0, mais **la dernière valeur qui a circulé sur le bus** — une sorte d'écho électrique du dernier octet transporté. Certains jeux en dépendent (volontairement ou par accident). L'émulateur mémorise donc cette dernière valeur dans un registre appelé *MDR* (*Memory Data Register*) et la renvoie pour toute adresse non mappée. Oublier ce détail provoque des bugs bizarres et lointains, très difficiles à diagnostiquer.

5. Le cœur : le processeur 65C816

Le CPU exécute la logique du jeu : « si le joueur appuie à droite, avancer Mario ». Il ne fait rien de magique — il répète sans cesse un cycle de trois temps, des millions de fois par seconde.

1• Lire l'instruction (fetch) → 2• La décoder → 3• L'exécuter ↻

Les registres du 65C816

Un registre est une case ultra-rapide interne au processeur. Le 65C816 en possède une poignée :

Registre	Rôle
A	Accumulateur — où se font les calculs (additions, comparaisons...).
X, Y	Index — servent à parcourir des tableaux, compter des boucles.
S	Pointeur de pile — mémorise où revenir après un sous-programme.
PC	Compteur de programme — l'adresse de la prochaine instruction.
P	Registre d'état — un octet de <i>drapeaux</i> (résultat nul ? négatif ? retenue ?).
DBR, PBR	Banques de données et de programme — les 8 bits hauts des adresses 24 bits.
D	Registre de page directe — une base pour un mode d'adressage rapide.

La particularité 8/16 bits

Le 65C816 descend du 6502 (8 bits) qui équipait la NES. Il démarre en **mode émulation**, où il se fait passer pour un 6502, puis un jeu le bascule en **mode natif** 16 bits dès les premières instructions. Mieux : deux drapeaux du registre **P** (nommés **M** et **X**) permettent au jeu de choisir **à la volée** si l'accumulateur et les index travaillent sur 8 ou 16 bits. Une même instruction change donc de comportement selon le contexte — l'émulateur teste ces drapeaux à chaque opération concernée.

Les modes d'adressage

Une instruction comme « charger l'accumulateur » a besoin de savoir *d'où* lire la valeur. Le 65C816 offre 24 **modes d'adressage** pour calculer cette adresse :

Exemple	Mode	Adresse calculée
LDA #\$1234	Immédiat	La valeur est dans l'instruction elle-même.
LDA \$2000	Absolu	Banque de données + \$2000 .
LDA \$2000,X	Absolu indexé	\$2000 + contenu de X.
LDA (\$10)	Indirect	Lit un pointeur en page directe, puis va à l'adresse pointée.
LDA [\$10],Y	Indirect long indexé	Pointeur 24 bits + Y — pour parcourir de gros tableaux.

Le cycle d'exécution, en code

```
// Le squelette d'un pas de CPU (simplifié)
fn step(cpu, bus) {
    if une_interruption_est_en_attente() {
        servir_l_interruption(); // NMI (vsync) ou IRQ (timer)
        return;
    }
    let opcode = bus.read(cpu.pbr, cpu.pc); // 1. fetch
    cpu.pc += 1;
    match opcode { // 2. décode (256 cas)
        0xA9 => { let v = lire_immediat(cpu, bus); cpu.a = v; maj_drapeaux(cpu, v); } // LDA
#
        0x8D => { let adr = lire_absolu(cpu, bus); bus.write(adr, cpu.a); } // STA
abs
        0x4C => { cpu.pc = lire_mot(cpu, bus); } // JMP
        // ... les 253 autres opcodes ...
    }
}
```

Notre implémentation couvre les **256 instructions**, tous les modes d'adressage, l'arithmétique décimale (BCD, utilisée pour afficher scores et chronos), les instructions de pile et de saut, les déplacements de blocs mémoire, et les instructions spéciales comme **WAI** (met le CPU en veille jusqu'à la prochaine interruption) ou **STP** (arrêt total).

Les interruptions

Une **interruption** est un signal qui force le CPU à suspendre ce qu'il fait pour exécuter un sous-programme spécial, puis reprendre. La SNES en a deux principales :

- **NMI** (*Non-Maskable Interrupt*) : déclenchée à chaque fin d'affichage d'image (le *V-blank*). C'est le **battement de cœur** du jeu — il y met à jour l'écran, lit les manettes, avance la musique. 50 ou 60 fois par seconde.
- **IRQ** : déclenchée à une ligne/position d'écran choisie par le jeu, pour des effets « à mi-écran » (une barre de statut fixe pendant que le décor défile, par exemple).

Servir une interruption suit une séquence précise : le CPU sauvegarde sur la pile la banque de programme, l'adresse courante et les drapeaux, met le drapeau d'interruption, puis saute à une adresse fixe (le **vecteur**) où le jeu a placé son gestionnaire. Reproduire cette séquence au cycle près est essentiel : beaucoup de jeux se calent précisément dessus.

🔑 À retenir

Le temps se compte en cycles. Chaque lecture ou écriture en mémoire fait avancer l'« horloge maître » d'un nombre précis de cycles, qui dépend de la zone visée : la ROM rapide, la RAM, les registres, la zone manettes... n'ont pas la même vitesse d'accès. C'est ainsi que l'émulateur sait *combien de temps* a passé, et donc quand déclencher les événements vidéo et audio.

6. Le chef d'orchestre : le timing et l'ordonnanceur

La SNES bat au rythme d'une horloge maître d'environ **21,5 MHz** (21,5 millions de cycles par seconde). L'image se construit ligne par ligne, comme un pinceau qui balaie l'écran de gauche à droite et de haut en bas.

	NTSC (Amérique/Japon)	PAL (Europe)
Horloge maître	21 477 272 Hz	21 281 370 Hz
Lignes par image	262	312
Images par seconde	≈ 60,1	≈ 50,0
Image visible	256 × 224 pixels (lignes 1 à 224)	

Chaque ligne dure 1364 cycles maîtres. Après la dernière ligne visible commence le **V-blank** : le « pinceau » remonte en haut de l'écran. C'est le seul moment sûr pour modifier la mémoire vidéo — d'où la NMI qui prévient le CPU à cet instant.

L'ordonnanceur (scheduler)

Comment savoir, à tout moment, quel événement doit se produire (fin de ligne, NMI, IRQ, transfert automatique) sans tout ralentir ? L'émulateur utilise une astuce : il tient un seul compteur d'horloge (un entier 64 bits) et un unique « **prochain événement** ».

```
struct Scheduler {
    clock:      u64,    // cycles écoulés depuis l'allumage
    next_event: u64,    // horodatage du prochain événement
    v:          u16,    // ligne courante (0..261 ou 0..311)
}
```

À chaque accès mémoire du CPU, on ajoute le coût en cycles, puis on compare l'horloge à `next_event`. Tant que l'horizon n'est pas franchi, on ne fait rien — une seule comparaison. Quand il l'est, on traite l'événement (dessiner la ligne, déclencher NMI/IRQ, faire avancer l'audio) et on calcule le prochain horizon. Ce mécanisme est ce qui rend l'émulation à la fois **exacte** et **rapide**.



Pour aller plus loin

La SNES supporte deux régions : PAL (Europe, 50 Hz) et NTSC (60 Hz). Un jeu européen tourne plus lentement mais affiche plus de lignes. L'émulateur détecte la région dans l'en-tête de la cartouche et adapte l'ordonnanceur et la cadence d'affichage : vos trois jeux sont PAL, donc rendus à 50 images par seconde, exactement comme sur une SNES européenne branchée à une télé de l'époque.

7. La puce graphique : le PPU

Le PPU est le composant le plus riche — et le plus délicat à émuler fidèlement. Son rôle : transformer des **tuiles** et des **sprites** rangés en mémoire vidéo en une image de 256 × 224 pixels, soixante fois par seconde.

Trois mémoires vidéo

- **VRAM** (64 Ko) : les *tuiles* (petits carrés de 8×8 pixels) et les *plans de tuiles* (quelle tuile à quel endroit).
- **CGRAM** (256 couleurs) : la palette. Chaque pixel ne stocke pas une couleur, mais un *numéro* de couleur dans cette palette. Chaque couleur est codée sur 15 bits (5 bits de rouge, 5 de vert, 5 de bleu).
- **OAM** (544 octets) : la table des sprites — position, tuile, priorité de chacun des 128 objets mobiles.

💡 Analogie

Imaginez un tableau de feutrine avec des gommettes. Les **tuiles** sont un stock de petits motifs réutilisables ; le **plan de tuiles** dit où coller chaque motif ; la **palette** dit quelles couleurs donner aux motifs. Redessiner l'écran, c'est relire cette recette 50 à 60 fois par seconde. Réutiliser les tuiles économise énormément de mémoire — indispensable en 1990.

Comment une tuile est stockée : les plans de bits

Une tuile 4 bits (16 couleurs) fait 8×8 pixels, chacun un numéro de 0 à 15. Ce numéro est découpé en 4 **plans de bits** : le plan 0 contient le bit de poids faible de chaque pixel, le plan 1 le bit suivant, etc. Pour reconstruire un pixel, on rassemble un bit de chaque plan :

```
// pixel (x=0) d'une ligne, à partir de 4 octets de plans
val = (plan0 >> 7 & 1)
    | (plan1 >> 7 & 1) << 1
    | (plan2 >> 7 & 1) << 2
    | (plan3 >> 7 & 1) << 3; // -> numéro de couleur 0..15
```

Se tromper dans l'ordre des plans ou des bits produit une image « en rayures » caractéristique — un piège classique. Cet émulateur a justement rencontré ce symptôme en développant le SuperFX (voir §10).

Les couches de fond et les huit modes

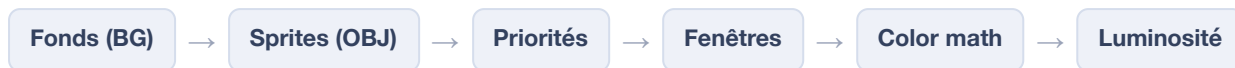
Le PPU superpose jusqu'à 4 **couches de fond** (BG1–BG4). Un registre (\$2105) choisit le **mode** (0 à 7), qui décide combien de couches existent et avec combien de couleurs :

Mode	Couches et profondeur	Usage typique
0	4 couches, 4 couleurs chacune	Menus simples
1	BG1/BG2 16 couleurs, BG3 4 couleurs	Le plus courant (plateformes, RPG)
2	2 couches 16 couleurs + décalage par colonne	Effets de profondeur
3	BG1 256 couleurs, BG2 16 couleurs	Écrans riches (menus, titres)
5/6	Haute résolution (512 points de large)	Textes fins, écrans de saisie
7	1 couche, rotation + zoom	Mode 7 : circuits, cartes du monde

Le rendu ligne par ligne

Notre PPU dessine **une ligne à la fois**, à la fin de chaque ligne visible. C'est essentiel : comme le jeu peut changer les registres vidéo *entre* deux lignes, dessiner ligne par ligne reproduit naturellement

presque tous les effets d'écran. Pour chaque ligne, le pipeline est :



Sprites (OBJ)

Jusqu'à 128 objets mobiles, de tailles variées. Le PPU a des limites matérielles strictes : au plus **32 sprites et 34 tuiles par ligne**. Au-delà, les sprites en trop disparaissent — un phénomène de clignotement que les joueurs de l'époque connaissaient bien, et que l'émulateur reproduit fidèlement (drapeaux de dépassement inclus).

Priorités

Les **priorités** décident qui passe devant qui — Mario devant le décor mais derrière un nuage. Chaque mode a sa propre table d'ordre de superposition entre les couches et les 4 niveaux de priorité des sprites. Une erreur ici et les personnages passent derrière le décor.

Fenêtres et color math

Les **fenêtres** masquent des zones rectangulaires — le rond de lumière d'une torche dans une pièce sombre. La **color math** additionne ou soustrait deux « écrans » (principal et secondaire) pour créer transparences, ombres et fondus. Par exemple, une eau semi-transparente : chaque canal de couleur est moyenné entre le décor et l'eau. C'est cette color math qui allume le menu de sélection de *Super Mario All-Stars* (superposition de deux couches) — sans elle, l'écran resterait noir.

Le Mode 7 : rotation et zoom

Le Mode 7 est à part : une seule couche, mais qu'il transforme par une matrice mathématique appliquée **à chaque pixel**. Pour le pixel écran (x, y), la position dans la texture est :

```
tex_x = A · (x + H) + B · (y + V) + X0  
tex_y = C · (x + H) + D · (y + V) + Y0
```

où A, B, C, D forment la matrice (en virgule fixe 8.8), et H/V/X0/Y0 les décalages et le centre. En faisant varier la matrice **à chaque ligne** (via le HDMA, §8), on obtient la perspective des circuits de F-Zero : le sol semble filer vers l'horizon.



Pour aller plus loin

Fonctionnalités fines. Le PPU gère aussi : la *mosaïque* (pixellisation pour les transitions), le *décalage par tuile* (offset-per-tile, où une couche fournit un décalage de défilement différent pour chaque colonne de 8 pixels), la *haute résolution* (512 points, pour les textes fins des écrans de saisie de nom), et l'*entrelacement* (448 lignes). Chacune est un petit monde de règles que l'émulateur reproduit d'après la documentation matérielle.

8. Le déménageur : DMA et HDMA

Copier des milliers d'octets vers la VRAM un par un avec le CPU serait trop lent. Le **DMA** (*Direct Memory Access*) le fait à la place, en bloc, à environ un octet tous les 8 cycles. Le jeu remplit quelques registres

(adresse source, registre de destination, taille) puis lance le transfert ; le CPU est mis en pause le temps de la copie. La SNES a **8 canaux** DMA indépendants.

Le **HDMA** (*H-blank DMA*) est plus malin : il transfère **un petit paquet à chaque ligne d'écran**, automatiquement, juste avant que la ligne soit dessinée. C'est ce qui permet :

- les **dégradés de ciel** — une couleur de fond différente par ligne ;
- la **perspective du Mode 7** — une matrice différente par ligne ;
- les **écrans ondulants** — un décalage horizontal par ligne (effet de chaleur, de vague).

🔑 À retenir

Détail qui fait tout : le HDMA d'une ligne doit s'exécuter **avant** le rendu de cette ligne. Se tromper d'une ligne, et le dégradé est décalé d'un cran. C'est l'une des sources de bugs les plus classiques d'un émulateur — d'où l'attention portée à l'ordre exact des événements dans l'ordonnanceur.

9. Le son : l'APU (SPC700 + S-DSP)

Surprise : la partie audio de la SNES est **un ordinateur à part entière**, avec son propre processeur (le **SPC700**), sa propre mémoire (64 Ko) et sa puce de synthèse (le **S-DSP**). Il ne partage rien avec le CPU principal, sauf **quatre boîtes aux lettres** (les ports `$2140-$2143`).

💡 Analogie

Le CPU principal et l'APU sont comme deux bureaux qui ne communiquent que par quatre cases postales. Au démarrage, un minuscule programme gravé dans l'APU (l'*IPL ROM*, 64 octets) répond « je suis prêt » (les octets `$AA`, `$BB`) ; le jeu lui envoie alors, case par case, tout son **moteur sonore** et ses instruments. La quasi-totalité des jeux *attendent* cette poignée de main avant de continuer — l'émuler est donc indispensable, même pour afficher la moindre image.

Comment naît un son

Le S-DSP mélange **8 voix** en un flux stéréo à 32 kHz. Pour chaque voix :

- **BRR** : les instruments sont des échantillons sonores compressés en blocs de 9 octets (le format *Bit Rate Reduction*). On les décompresse à la volée, en appliquant un petit filtre qui reconstruit la forme d'onde à partir des deux échantillons précédents.
- **Hauteur (pitch)** : on lit l'échantillon plus ou moins vite pour jouer une note grave ou aiguë, avec une interpolation gaussienne pour adoucir le résultat (une table de 512 coefficients).
- **Enveloppe ADSR** : le volume évolue dans le temps selon quatre phases — *Attack* (montée), *Decay* (chute), *Sustain* (maintien), *Release* (extinction). C'est ce qui distingue un piano (attaque vive, extinction lente) d'un orgue (volume constant).
- **Bruit** : un générateur pseudo-aléatoire, pour les explosions et vagues.
- **Écho** : un tampon circulaire en mémoire, relu avec un délai et un filtre à 8 coefficients, recrée la réverbération si caractéristique des musiques de la SNES — pensez aux grottes de *Secret of Mana*.

Pour aller plus loin

Synchronisation et son de l'ordinateur. L'émulateur fait tourner le SPC700 « en rattrapage » : le bus le fait avancer juste avant chaque accès aux boîtes aux lettres et une fois par ligne d'écran, avec une conversion d'horloge en virgule fixe (32.32 bits) pour ne jamais dériver. Les échantillons à 32 kHz produits sont ensuite **ré-échantillonnés** vers la fréquence de la carte son de l'ordinateur (souvent 44,1 ou 48 kHz), avec un léger ajustement dynamique du débit pour absorber la différence entre l'horloge émulée et celle du PC. Résultat : une éventuelle dérive devient un imperceptible vibrato plutôt qu'un craquement. La fidélité audio a d'ailleurs été vérifiée en analysant le signal produit (énergie, présence de l'écho) — pas seulement à l'oreille.

10. Les puces de cartouche : le coprocesseur SuperFX

Certaines cartouches embarquaient leur propre puce pour dépasser les limites de la console. La plus célèbre est le **SuperFX** (aussi appelé GSU), un véritable second processeur, qui faisait tourner la 3D de *Star Fox* et les décors de *Yoshi's Island*.

Qu'est-ce que le GSU

Le GSU est un processeur **RISC 16 bits** à ~10 MHz, avec 16 registres et un jeu d'instructions entièrement différent du 65C816. Il vit dans la cartouche, accède à la ROM et à une RAM dédiée (*Game Pak RAM*), et sait **dessiner des pixels** (instruction **PL0T**) dans un tampon que le jeu récupère ensuite pour l'afficher. Le CPU principal et le GSU se partagent la ROM/RAM à tour de rôle : pendant que le GSU calcule, le CPU attend.

L'émuler, c'est écrire **un CPU complet de plus** — le vrai défi de ce projet. Le GSU de cet émulateur exécute réellement les programmes de *Yoshi's Island*, décompresse ses graphismes et affiche l'écran de sélection de la langue.

Histoire d'un bug

Un octet qui changeait tout. Pendant des jours, *Yoshi's Island* restait obstinément noir : le GSU tournait, mais son décompresseur bouclait sans fin. Après avoir construit un outil de trace dédié et éliminé une à une plusieurs pistes (format des pixels, décodage des banques mémoire, tampon de lecture), la cause s'est révélée être un **décalage d'un seul octet sur le compteur de programme R15** : notre code avançait R15 dès la lecture de l'instruction, alors que le vrai GSU ne l'avance qu'à la lecture des opérandes. Toutes les valeurs calculées à partir de R15 (cibles de saut, adresses) étaient donc fausses d'un octet, ce qui donnait de mauvais compteurs de boucle et figeait la décompression. Une ligne de correction, et le jeu s'est mis à booter. C'est l'essence du travail d'émulation : des bugs minuscules aux conséquences spectaculaires, que seule une comparaison méticuleuse avec le matériel de référence permet de trouver.

11. La boucle d'émulation : tout synchroniser

Assembler tout cela tient, au fond, en une boucle. Pour produire une image :

```
fn run_frame(inputs) -> Image {
    appliquer_les_manettes(inputs);
    while !fin_de_trame {
        cpu.step(bus); // le CPU exécute une instruction...
        // ...chaque accès mémoire a fait avancer l'horloge, ce qui
        // déclenche AU BON MOMENT, via l'ordonnanceur :
        // • le rendu d'une ligne d'écran,
        // • les interruptions NMI / IRQ,
        // • les transferts HDMA,
        // • le rattrapage du processeur audio.
    }
    return framebuffer; // l'image de 256x224 est prête
}
```

Le frontend appelle `run_frame` à la cadence de la région (50 ou 60 fois par seconde), convertit l'image en pixels à l'écran (via une fenêtre `winit` et la bibliothèque `pixels`), envoie le son à la carte audio (`cpal`), et transmet l'état du clavier comme s'il s'agissait d'une manette. Le rythme est tenu par une **échéance absolue** : on calcule l'instant idéal de la prochaine image et on attend jusque-là, pour un défilement parfaitement régulier.

12. Au-delà de la console : sauvegardes et save states

Parce que l'émulateur **possède la totalité de l'état** de la machine, il peut faire des choses impossibles sur le vrai matériel :

- **Sauvegardes de cartouche (SRAM)** : la pile de sauvegarde de la vraie cartouche est simplement un fichier `.srm` à côté de la ROM, chargé au lancement et réécrit en quittant.
- **Save states** : on *séréalise* l'intégralité de l'état — CPU, PPU, APU, toutes les mémoires — dans un fichier. Le recharger fait reprendre le jeu **exactement** où il en était, au pixel et à l'échantillon près. Cette exactitude est vérifiée automatiquement : après sauvegarde puis rechargement, l'émulation reste identique bit pour bit.
- **Outils de débogage** : traces d'instructions, captures d'écran, inspection de la mémoire — les instruments qui ont permis de construire et de valider l'émulateur.



Pour aller plus loin

Le déterminisme, garant de justesse. Un émulateur bien écrit est *déterministe* : à partir du même état et des mêmes entrées, il produit toujours exactement la même suite d'images et de sons. C'est cette propriété qui rend les save states fiables, et qui permet de tester l'émulateur en comparant ses sorties à une référence connue.

13. Comment cet émulateur a été construit et vérifié

Émuler fidèlement une console suppose une méthode aussi rigoureuse que le matériel lui-même. Trois principes ont guidé ce projet :

1. Ne jamais inventer une constante matérielle

Chaque coût en cycles, chaque bit de registre, chaque table de priorités provient d'une **documentation de référence vérifiée** (les bases de connaissances de la communauté, et le code source d'émulateurs de référence). Une constante inventée « au jugé » est la première cause de bugs subtils.

2. Vérification adversariale

Chaque composant, une fois écrit, est **relu par un « avocat du diable »** dont le but est de trouver les écarts avec le matériel réel — pas d'approuver le code. Cette relecture croisée a débusqué quantité de petites erreurs avant qu'elles ne deviennent des bugs visibles.

3. Validation par jalons sur de vrais jeux

Le développement a progressé par **jalons vérifiables** : d'abord « le CPU démarre », puis « le son répond », puis « les fonds s'affichent », « les sprites », « la manette pilote le jeu », « la musique joue »... Chaque jalon était prouvé sur un vrai jeu commercial, avec une preuve concrète (capture d'écran, analyse du son, trace d'exécution). Choisir les jalons pour qu'ils testent *un* composant à la fois rend les bugs attribuables.

À retenir

Le résultat : environ 16 000 lignes de Rust, plus de 250 tests automatiques, un émulateur qui fait tourner plusieurs jeux commerciaux (image, son, sauvegardes, save states) et même un jeu à puce SuperFX — le tout empaqueté dans une application macOS double-clicable avec barre de menus et sélecteur de jeu.

14. Glossaire

Bit / Octet	Unité de base (0 ou 1) / groupe de 8 bits (0–255).
Registre	Petite case mémoire interne à une puce ; y écrire commande la puce.
MMIO	Adresses mémoire qui, au lieu de RAM, pilotent une puce (registres mappés en mémoire).
Open bus / MDR	Valeur « fantôme » renvoyée par une lecture d'adresse non mappée : le dernier octet vu sur le bus.
V-blank	Intervalle après la dernière ligne visible, seul moment sûr pour modifier la vidéo.
NMI / IRQ	Interruptions : signaux qui détournent le CPU vers un gestionnaire (V-blank / minuterie).
Tuile	Motif de 8×8 pixels, brique de base des images de la SNES.
Sprite / OBJ	Objet mobile (personnage, projectile), positionné librement à l'écran.
Palette / CGRAM	Table des couleurs ; les pixels stockent un numéro de couleur, pas la couleur.
DMA / HDMA	Transferts rapides d'octets en bloc / par ligne d'écran.
ADSR	Enveloppe de volume d'un son (Attaque, Chute, Maintien, Extinction).
BRR	Format d'échantillon audio compressé de la SNES (blocs de 9 octets).
Mode 7	Mode graphique à rotation et zoom d'une couche unique.
GSU / SuperFX	Coprocasseur dans certaines cartouches, un CPU RISC qui dessine des pixels.
Save state	Photographie complète de l'état de la machine, rechargeable à l'identique.

Document généré pour accompagner l'émulateur SNES écrit en Rust · CPU 65C816, PPU (modes 0–7, color math, HDMA, offset-per-tile, hires), APU (SPC700 + S-DSP), DMA/HDMA, coprocasseur SuperFX, save states · ≈ 16 000 lignes, 250+ tests. Édition détaillée v2.0.